

J-Lens on a looped model: the cheap baseline wins early

MATS 12.0 application · Jan Kirin · September 2026

Executive summary

I keep coming back to what hidden states can tell us before a model answers. Here I tested J-Lens on Ouro-2.6B: 48 shared layers, run four times. Does it read intermediates better than the logit lens? Do lenses aimed at local and final exits converge? I expected more agreement. Also more winning.

On 90 clean multihop items, the logit lens beats the final-exit Jacobian lens at loops 1–3. With 100 fitting prompts, the Jacobian-minus-logit differences are -0.356 , -0.479 , -0.407 and -0.022 across the four loops. These are differences in excess hit@10: finding the intermediate somewhere in a loop, minus finding matched control names the same way. The first three paired 95% intervals are below zero; the last overlaps zero. Arithmetic is less tidy: on 51 items, J-Lens loses at loop 1 but wins at loop 2 ($+0.140$, interval $[+0.020, +0.264]$). So “J-Lens never wins” is out.

The second result: lenses fitted to each loop's own exit disagree strongly with the lens fitted to the final exit. Across all 148 stimuli and 48 layers, mean $KL(\text{local} \parallel \text{eventual})$ is 4.77, 4.10 and 6.30 nats at loops 1–3. Their top-1 agreement over the last eight layers is at most 3.3%. This measures disagreement between lenses; it does not establish what the model is actually thinking. My expected steady convergence fails at this position. One token earlier, KL does decrease, although top-1 agreement stays around 0.5% or less. Annoying, useful control.

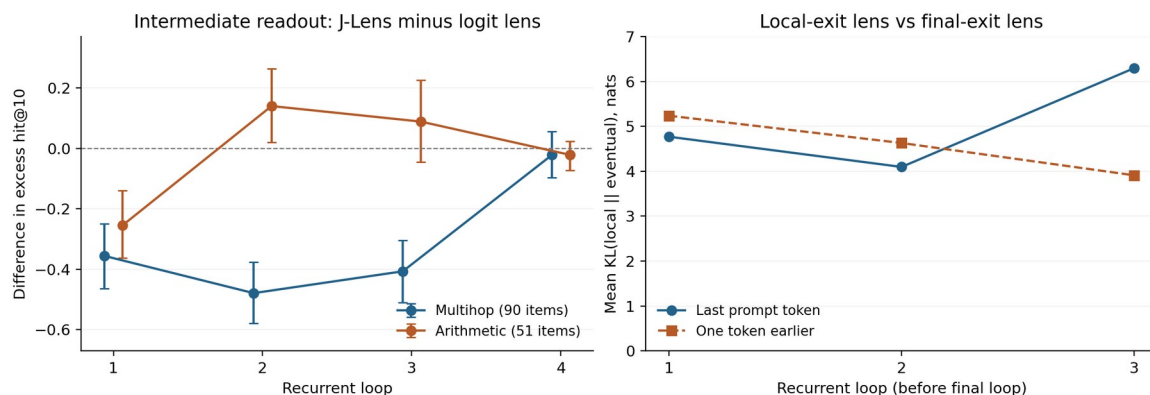


Figure 1. B300 100-prompt fits. Left: paired differences with unadjusted 95% item-bootstrap intervals; below zero favours the logit lens. Right: lens-to-lens KL, averaged over 148 stimuli and 48 layers; evaluation run locally using B300-fitted lenses. The depth trend changes with token position.

My guess is that Ouro's shared-head supervision makes the logit lens unusually competitive. I have not isolated that mechanism.

The local recorder passes 18/18 bit-exact checks. Audits caught inflated control scores, probe leakage and a matcher accepting “11” for “1”. Fixed. Limits: one model, small nested fits, unadjusted intervals, and no demonstrated causal intermediates. Next: a tuned lens and a matched shared-head ablation.

What I ran

Model revision: 1ed04250; released J-Lens code: 581d3986. I recorded residual states at all 192 loop/layer locations and fitted prompt-averaged Jacobian maps on Wikitext, then applied the model's output readout. The logit lens applies that readout directly.

The released stimuli contain 93 multihop and 55 order-of-operations items. Excluding leaked or unscorable intermediates leaves 90 multihop items (100 intermediate slots) and 51 numeric arithmetic items. At the last prompt token, each intermediate and each same-kind control gets the same best-over-48-layers hit@10 test. I average slots within items, then items. The paired bootstrap resamples items: 20,000 draws, seed 0, eight unadjusted 95% intervals.

J-Lens minus logit lens	Loop 1	Loop 2	Loop 3	Loop 4
Multihop	-0.356	-0.479	-0.407	-0.022
95% interval	[-0.463, -0.249]	[-0.579, -0.377]	[-0.510, -0.304]	[-0.096, +0.056]
Arithmetic	-0.255	+0.140	+0.089	-0.021
95% interval	[-0.362, -0.139]	[+0.020, +0.264]	[-0.045, +0.226]	[-0.072, +0.024]

All four 100-prompt lenses were fitted on the B300. The final-exit and fit-size evaluations were retained from the pod. Its all-exit results did not publish before timeout, so those evaluations were rerun locally on the 5070 Ti using the retained B300 lenses. Both local readout positions passed strict provenance analysis. The pod's final reporting pipeline did not finish.

Five examples, including the awkward ones

These are the existing seed-0 random sample from 142 eligible items in the older local evaluation, not the B300 run. The full prompts and per-loop ranks are in APPLICATION_EXAMPLES.md. "Intermediate" is the dataset label, not proof that the model used it.

Prompt, abbreviated	Intermediate → target	Model continuation
Chemical symbol of element 26	iron → Fe	Fe.
One more than the number rhyming with hive	five → 6	100.
10 + 5 - 3 =	15 → 12	12
((1 + 2) * 3) - 4 =	9 → 5	9 ...
ten minus two times three equals	6 → four	Answer:

For the iron item, J-Lens's best vocabulary rank in loop 1 is 979; the logit lens's is 0. For the nested arithmetic item, "reading 9" can simply mean reading the model's wrong answer. Across all 148 stimuli, the boundary-aware correctness check accepts 72 continuations.

Baselines and things that broke

A separate local probe experiment uses $(a + b) * c$, with 17 possible intermediate labels. Mirror pairs stay in the same fold; after excluding labels absent from a training fold, 576 prompts remain in 39 unordered-pair clusters. On held-out data, loop-1 accuracy is 0.594 for the supervised probe, 0.764 for the logit lens and 0.392 for J-Lens (majority baseline 0.125). The probe reaches 0.356 at loop 3 versus the logit lens's 0.179. This is a separate task and an older lens whose fitting provenance is incomplete.

The most consequential bug was taking the maximum after averaging control names, while giving each true intermediate its own maximum. That inflated the excess score. The corrected calculation gives controls the same treatment. Agent review also caught the mirror-pair leak and unsupported claims about Jacobian norms; the old norm interpretation was retracted. I also replaced prefix matching that treated "11" as an answer of "1".

More fitting data did not rescue early multihop readout over the tested 8/32/56/80/100 ladder. These are nested prefixes, so they cannot tell me run-to-run variance. I still need fixed-size replicate fits, the tuned-lens baseline, and a matched comparison of position-summed versus per-position Jacobians and alternative averaging methods.

The overnight bit

The plan was to rent a B300, sleep, and wake up to results. I left Claude Opus overseeing it. Both I and the session went to sleep. The pod did not. About \$52 later, nothing was retrieved.

After rebuilding the controller, the rerun took nine launches. Nine. The successful one initially needed about 29 minutes per fitting prompt: 192 PyTorch threads were not helping. Capping them at eight brought that down to roughly 40 seconds. The B300 and I are on speaking terms now. Barely.

My estimate is 15 active hours, excluding rental setup, waiting on fits and the form. This write-up happened during the final overnight run. Not recommended. Not the plan either.

I used Claude Code and Codex for implementation, analysis, review and editing, including this rewrite. Three agent audits are retained. I did not manually recompute every statistic.

Code: <https://github.com/VykosMolt/Hidden-State-Evaluator>. Retained run identifier: jlens-b300-20260905-0359. Detailed numbers and artifact locations: B300_RESULTS_2026-09-05.md.